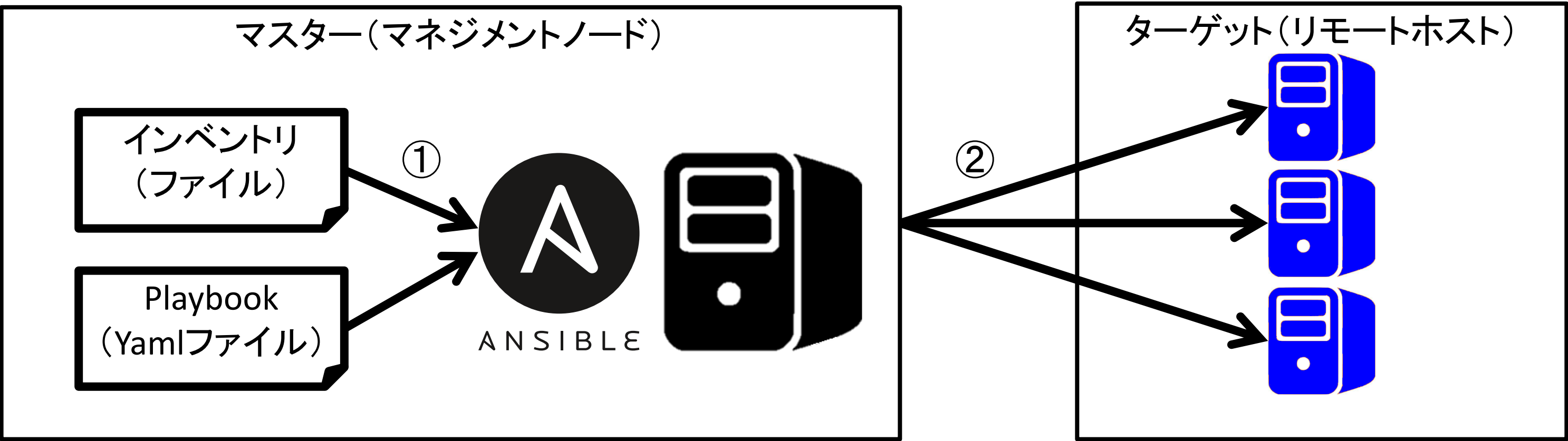


IaCハンズオン(理論編)

Ansibleの仕組み

Ansibleの仕組み

・Ansibleの基本的な仕組みは以下の通り



ステップ	概要
①入力ファイル読込	インベントリを元に、どのターゲットに対してメンテナンスを行うか、情報(IPアドレス等)を取得する Playbookを元に、ターゲットに対して行うメンテナンス内容(ソフトウェアの追加/変更/削除等)を取得する
②メンテナンス	①で取得した情報を元に、各ターゲットに対してメンテナンスを行う (Ansibleでできるのは、既存のターゲットに対するソフトウェアの変更のみで、 ハードウェアに対する変更(メモリ使用量の変更等)はサポートしていない。 ただし、情報取得はサポートしており、例えばメモリ使用量やOSのバージョン確認等は可能。)

Ansibleの仕組み

冪等性とは

- Ansibleの性質である冪等性の考え方は以下の通り

No	トピック	説明
1	冪等性とは？	▪ 同じ操作を何度繰り返しても、同じ結果が得られる性質 ▪ マスターからターゲットに対しあるファイルをコピーする例で言うと、Ansibleを何度実行しても、ターゲットにファイルがコピーされている状態は変わらない (実行するたびにファイル名がリネームされるとか、タイムスタンプが変わるとか起こらない)
2	冪等性を担保する仕組みは？	▪ Ansible側で同じ結果が得られるよう制御している ▪ No1と同じ例で言うと、Ansibleを1度実行してターゲットにファイルがコピー済みの場合、2回目の実行時はAnsible側でファイルがコピーされているかどうか判断し、コピー済みであればコピー処理はスキップするよう制御を行う
3	冪等性を採用する理由は？	▪ ターゲットの状態を気にしてPlaybookを書いたり実行したりする必要が無い (手間がかからない) ▪ No1と同じ例で言うと、ターゲットにファイルコピー済みならPlaybookから (リネームやタイムスタンプ変更等を気にして)コピー処理を外すとか、2回目の実行を控えるとか、する必要が無い (ファイルコピーされた状態にしたいのであれば、愚直に同じPlaybookを何度実行しても良い)

Ansibleの基本1ーインベントリー

Ansibleの基本1ーインベントリー

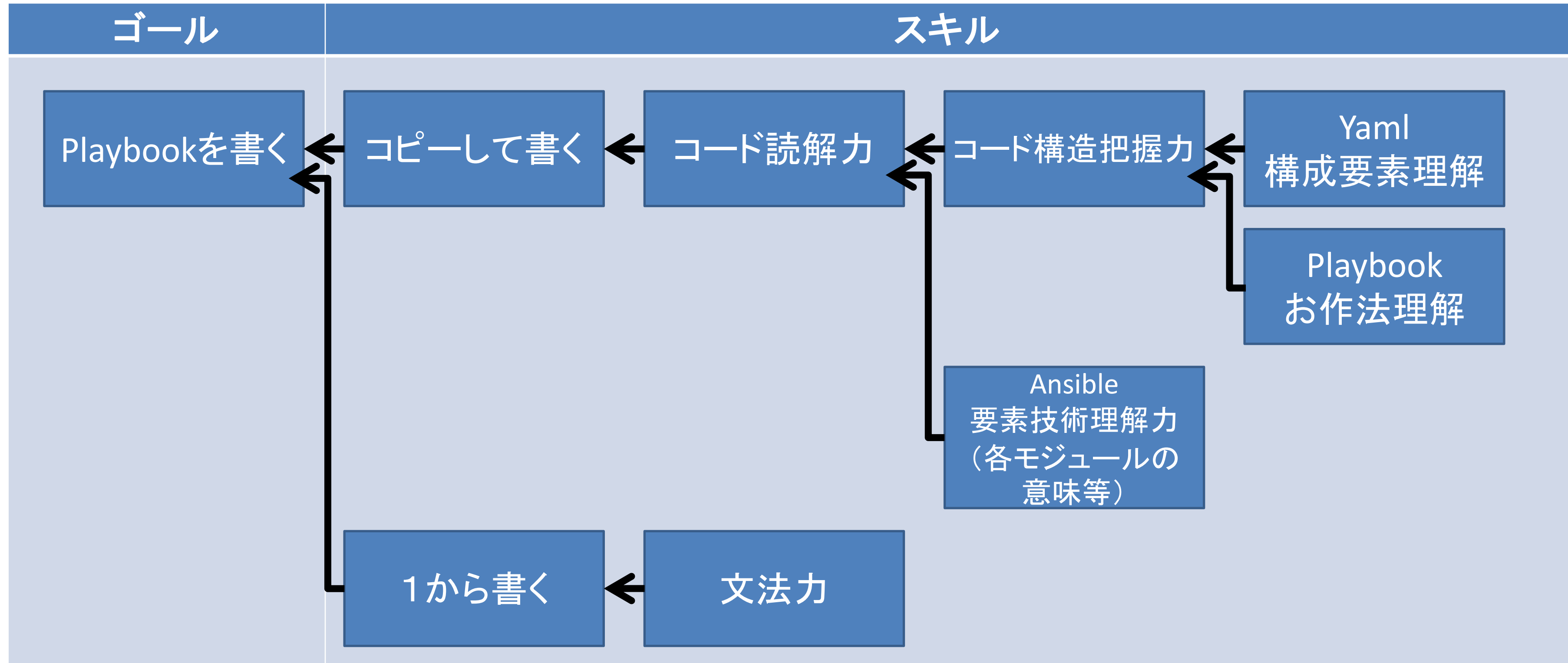
- Ansibleでメンテナンス対象のターゲットのホスト名やIPアドレス等を管理するためのファイルをインベントリと呼び、以下2種類が存在する

No	種類	概要
1	Hosts ファイル	▪ ほとんどのOSがホスト名をIPアドレスにマップするために使用するプレーンテキストファイル ▪ 拡張子は無い ▪ Ansible専用のファイルではなく、通常はDNS(ドメイン名システム)サーバーを使用せずにホスト名をIPアドレスに解決するために使用される ▪ Ansibleでは、No2の「インベントリファイル」が定義されない場合にターゲットの管理のためにデフォルトで使用されるため、「デフォルトインベントリ」と呼ばれる
2	インベントリ ファイル	▪ Ansibleでメンテナンス対象のターゲットを管理するためのファイル ▪ 拡張子は任意で、ini、yaml、json、txtなどがよく使用される(拡張子無しも可) ▪ インベントリとしてHostsファイルではなくインベントリファイルを使用するメリットは、より柔軟にターゲットを管理することができる点が挙げられる (Hostsの管理対象によらず、Ansible個別でターゲット管理ができる)

Ansibleの基本2ーPlaybook概要ー

Playbookを書くために必要なスキル分解

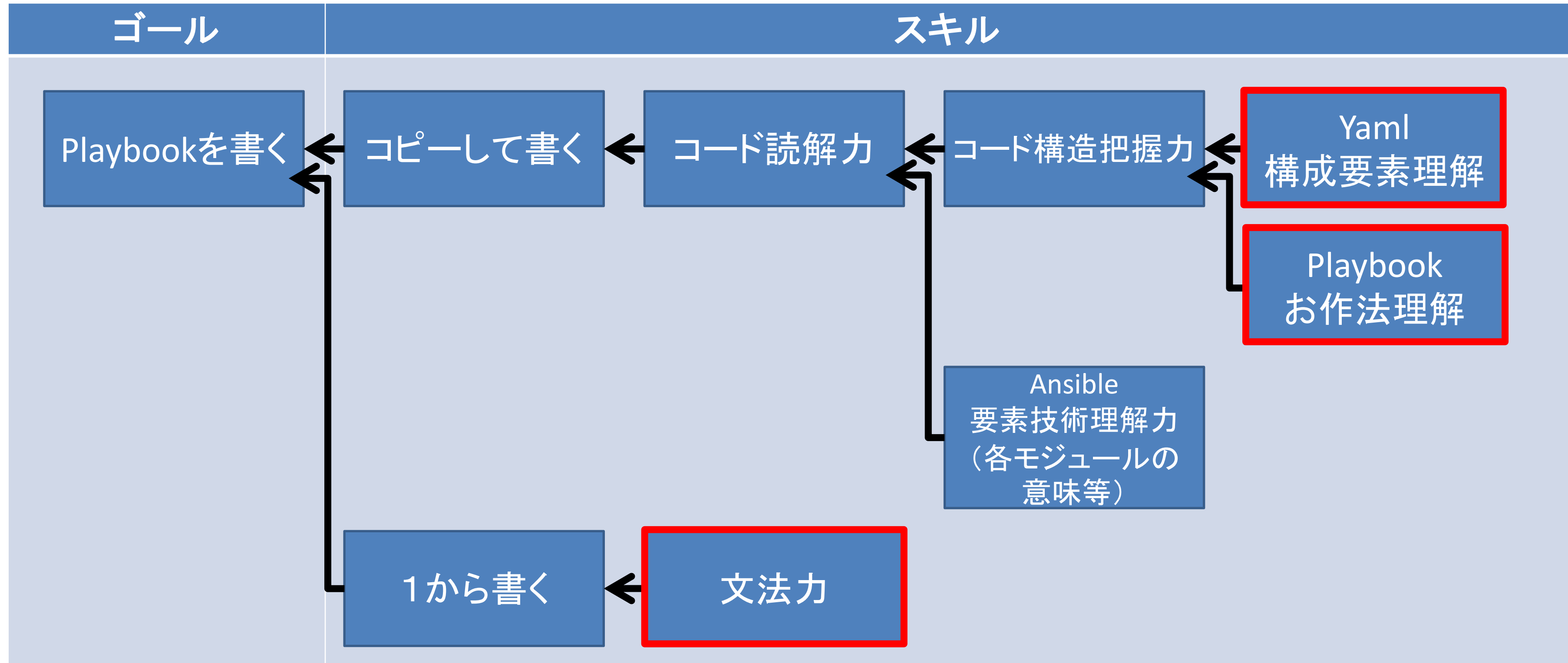
- ・Playbookを書くために必要なスキルは以下の通り



Ansibleの基本2ーPlaybook概要ー

本レクチャーで取り扱う内容

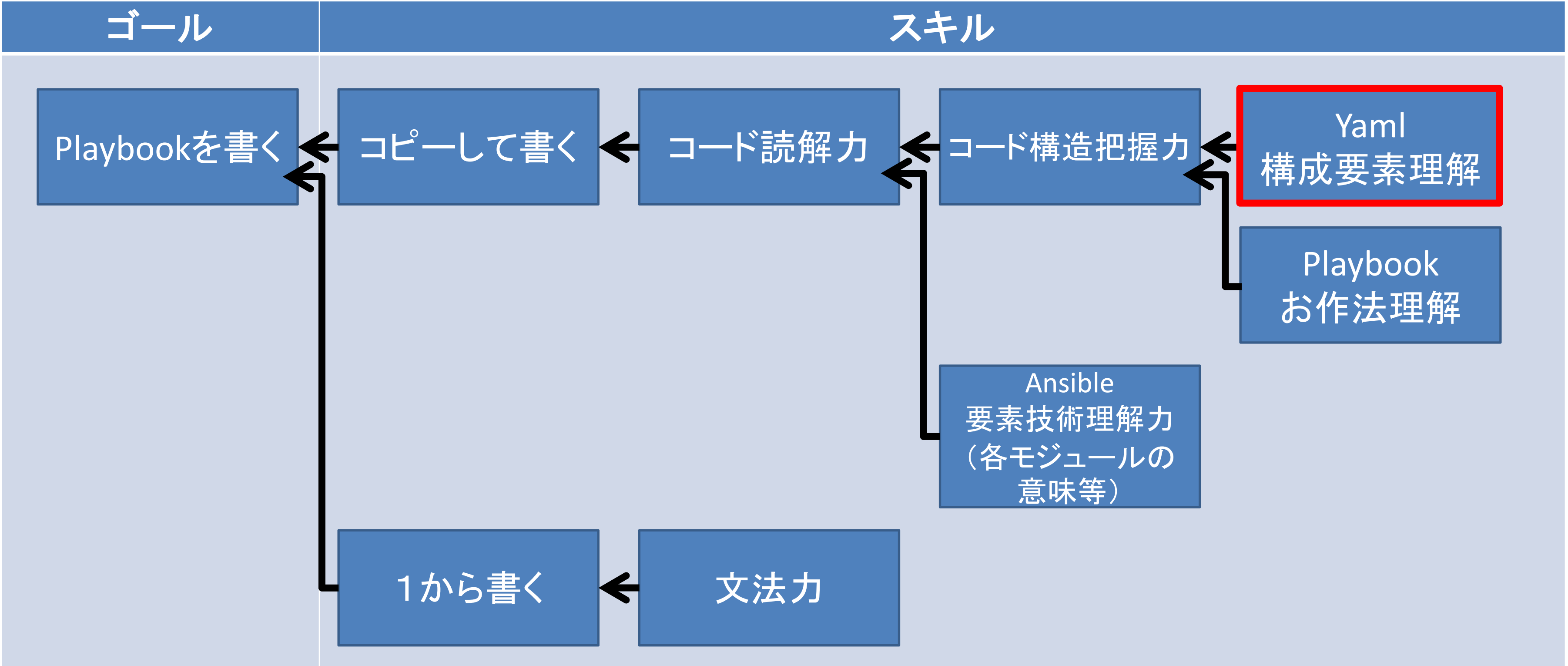
- ・本レクチャーで取り扱う内容は以下赤枠の通り



Ansibleの基本2ーPlaybook概要ー

Yaml構成要素

・コード読解力を付けるため、まずYaml構成要素について取り上げます



Ansibleの基本2ーPlaybook概要ー

Yaml構成要素

・Yamlの概要と構成要素は以下の通り

トピック		概要
Yaml概要		・2001年に開発された、データを人間が読めるように表現するための言語 ・XMLやJSONなどの言語に比べて、より簡潔で可読性が高いとされる ・データ表現形式には「リスト」と「ディクショナリー」の2つがある
構成要素	①リスト	＜表現方法（要素の列挙）＞ - A - B - C ＜具体例＞ - 玉ねぎ - にんじん - 肉

Ansibleの基本2ーPlaybook概要ー

Yaml構成要素

・Yamlの概要と構成要素は以下の通り

トピック		概要
構成要素	②ディクショナリー	<表現方法(キー:値形式)> A:aa B:bb C:cc <具体例> 玉ねぎ:1個 にんじん:2本 肉:300g

Ansibleの基本2ーPlaybook概要ー

Yaml構成要素

・Yamlの概要と構成要素は以下の通り

トピック		概要
構成要素	③リストと ディクショナリーの 組合せ	<表現方法(キー:値形式で要素を列挙)> - A:aa - B:bb - C:cc <具体例> - 玉ねぎ:1個 - にんじん:2本 - 肉:300g

Ansibleの基本2ーPlaybook概要ー

Yaml構成要素

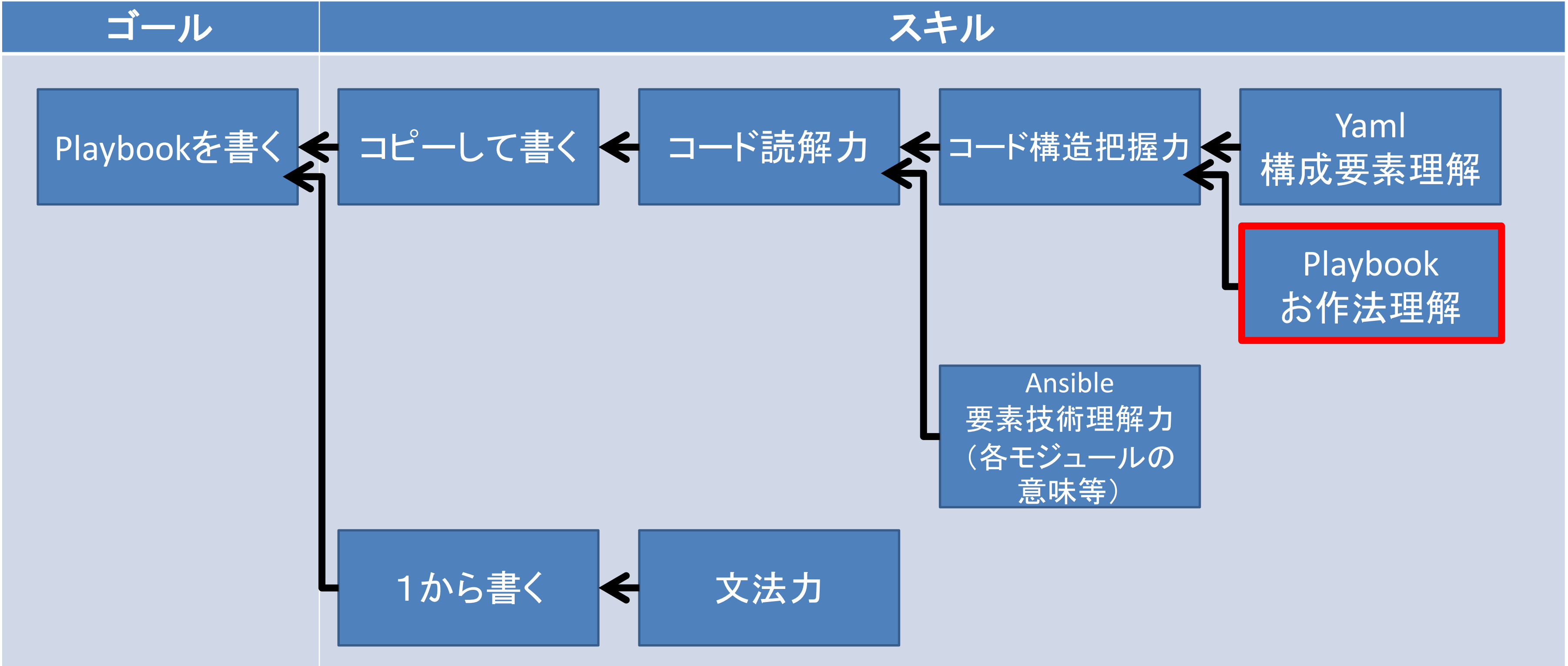
・Yamlの概要と構成要素は以下の通り

トピック	概要
構成要素のグループ化 (ラベル付け)	・各構成要素とも、データのグループ化のためにラベル付けをすることができる ＜リストのラベル付けの具体例＞ 材料: - 玉ねぎ - にんじん - 肉 - スライス ＜ディクショナリーのラベル付けの具体例＞ 材料: 玉ねぎ:1個 にんじん:2本 肉:300g スパイス:小さじ4

Ansibleの基本2ーPlaybook概要ー

Playbookお作法

・コード読解力を付けるため、次にPlaybookお作法について取り上げます



Ansibleの基本2ーPlaybook概要ー

Playbookお作法

・Playbookの概要とお作法は以下の通り

トピック	概要
Playbook概要	・ターゲットに対するメンテナンス内容を記述したファイル ・Yaml形式で記述し、ターゲットに対して実行するタスクを定義する ・タスクは、Ansibleで提供されるモジュールを使用して実行するコマンドやスクリプトを指す
お作法	＜お作法＞ ・サマリ部とタスク部の2つのセクションで構成する ・サマリ部の概要は以下の通り ・Playbookの概要を定義する(メンテナンス内容を簡潔に記載) ・メンテナンス対象のターゲットを定義する(変数を使用しインベントリを参照する) ・タスク部の概要は以下の通り ・ターゲットに対するメンテナンス内容を定義する ・メンテナンスの単位を「タスク」と呼びタスク定義には複数のタスクを定義できる ・各タスクはメンテナンス内容に応じて使用するモジュール(タスク実施の為の部品)を使用し定義する ・「name」という名称のキーを使用した、リストとディクショナリーの組合せで、直後に記載した構成要素をグループ化(ラベル付け)する

Ansibleの基本2ーPlaybook概要ー

Playbookお作法

- ・Playbookの概要とお作法は以下の通り

トピック	概要
お作法	<具体例> <pre># Playbookのyamlファイルサンプル - name: Setup nginx server on my list hosts: - devservers become: True tasks: - name: Install the latest version of nginx command: amazon-linux-extras install nginx1 -y - name: Start nginx service service: name: nginx state: started</pre>

Ansibleの基本2ーPlaybook概要ー

Playbookお作法

・「Yaml構成要素」、「Playbookお作法」両知識を動員した、コード構造把手順は以下の通り

手順	概要
①サマリ部と タスク部の セクションの 見極め	<具体例> <pre># Playbookのyamlファイルサンプル - name: Setup nginx server on my list hosts: - devservers become: True tasks: - name: Install the latest version of nginx command: amazon-linux-extras install nginx1 -y - name: Start nginx service service: name: nginx state: started</pre> サマリ部 タスク部

Ansibleの基本2ーPlaybook概要ー

Playbookお作法

・Playbookの概要とお作法は以下の通り

手順	概要
②各セクション内のグループの見極め	<具体例> <pre># Playbookのyamlファイルサンプル - name: Setup nginx server on my list hosts: - devservers become: True tasks: - name: Install the latest version of nginx command: amazon-linux-extras install nginx1 -y - name: Start nginx service service: name: nginx state: started</pre> グループ① グループ② グループ③

Ansibleの基本2ーPlaybook概要ー

Playbookお作法

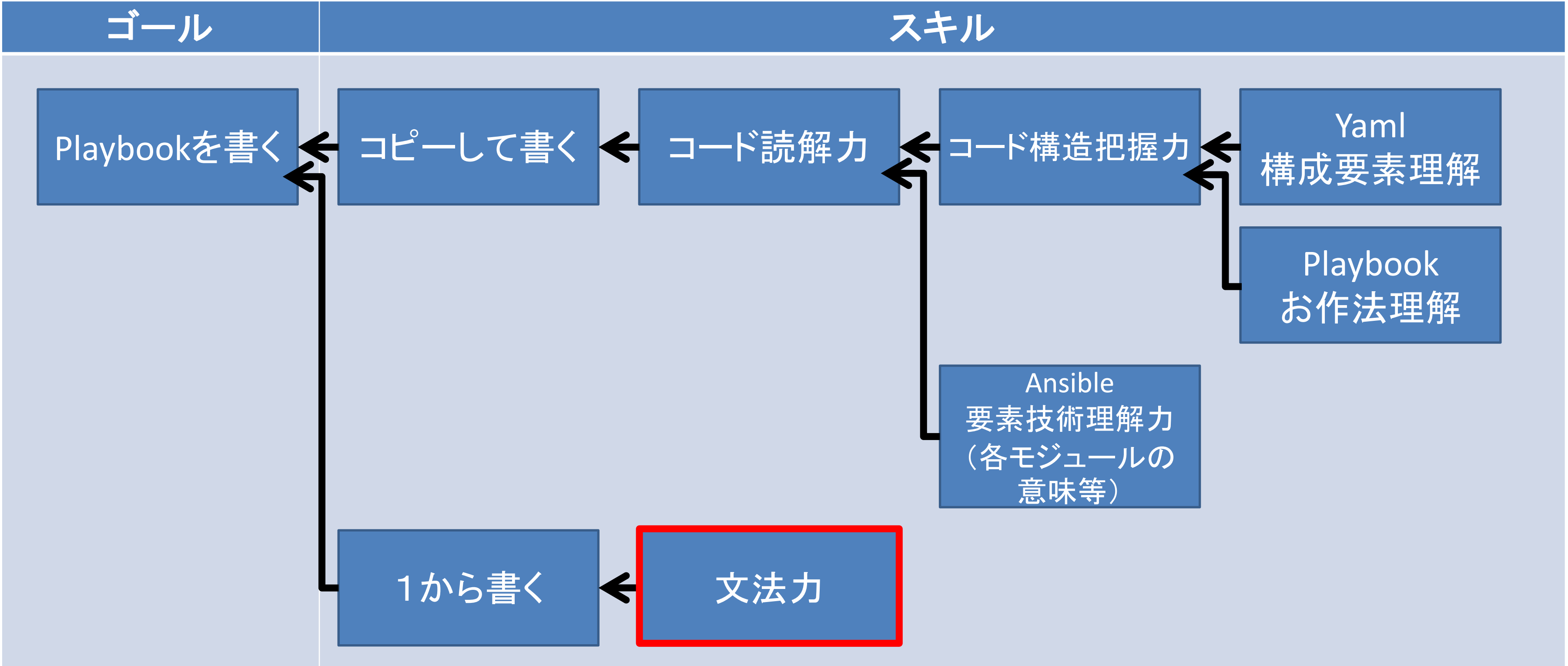
・Playbookの概要とお作法は以下の通り

手順	概要
③各グループ内の構成要素の見極め	<具体例> <pre># Playbookのyamlファイルサンプル</pre> <pre>- name: Setup nginx server on my list</pre><pre>hosts:</pre><pre> - devservers</pre><pre>become: True</pre> 構成要素① 構成要素② <pre>tasks:</pre> <pre>- name: Install the latest version of nginx</pre><pre> command: amazon-linux-extras install nginx1 -y</pre> 構成要素③ <pre>- name: Start nginx service</pre><pre> service:</pre><pre> name: nginx</pre><pre> state: started</pre> 構成要素④

Ansibleの基本2ーPlaybook概要ー

Yaml文法

・コード記述力を付けるため、最後にYaml文法について取り上げます



Ansibleの基本2ーPlaybook概要ー

Yaml文法

・Playbookを書くために最低限覚えるべき、Yaml文法8か条は以下の通り

No	文法の概要
1	リストを表すには、ハイフン(-)を使用して各要素を指定します。
2	ディクショナリーを表すには、キーと値をコロン(:)で区切ります。
3	ラベル付けの際はインデントを使用して構造を表現します。 各レベルは半角スペース2つでインデントされます。
4	コメントを表すには、#を使用します。
5	文字列を表すにはクォート(")で囲む必要があります。
6	真偽値を表すには、trueまたはfalseを使用します。
7	特殊文字を表すには、 エスケープシーケンスを使用する必要があります。 例えば、改行を表すには、¥nを使用します。
8	文字列を複数行で表すには、" "または">"を使用します。 " "を使用すると、改行が文字列に反映されます。 ">"を使用すると、改行が文字列から削除されます。

Ansibleの基本2ーPlaybook概要ー

Yaml文法

・Yaml文法8か条No1～4に対応する具体例は以下の通り

No	文法の概要
1	リストを表すには、ハイフン(-)を使用して各要素を指定します。
2	ディクショナリーを表すには、キーと値をコロン(:)で区切ります。
3	ラベル付けの際はインデントを使用して構造を表現します。 各レベルは半角スペース2つでインデントされます。
4	コメントを表すには、#を使用します。
5	文字列を表すにはクォート(")で囲む必要があります。
6	真偽値を表すには、trueまたはfalseを使用します。
7	特殊文字を表すには、 エスケープシーケンスを使用する必要があります。 例えば、改行を表すには、¥nを使用します。
8	文字列を複数行で表すには、" "または">"を使用します。 " "を使用すると、改行が文字列に反映されます。 ">"を使用すると、改行が文字列から削除されます。

Playbookのyamlファイルサンプル (文法8箇条No1~4に対応)

```
- name: Setup nginx server on my list
  hosts:
    - devservers
    - testservers
  tasks:
    - name: Install the latest version of nginx
      command: amazon-linux-extras install nginx1 -y
    - name: Start nginx service
      service:
        name: nginx
        state: started
```

No4

No1

No2

No3

Ansibleの基本2ーPlaybook概要ー

Yaml文法

・Yaml文法8か条No5～8に対応する具体例は以下の通り

No	文法の概要
1	リストを表すには、ハイフン(-)を使用して各要素を指定します。
2	ディクショナリーを表すには、キーと値をコロン(:)で区切ります。
3	ラベル付けの際はインデントを使用して構造を表現します。 各レベルは半角スペース2つでインデントされます。
4	コメントを表すには、#を使用します。
5	文字列を表すにはクォート(")で囲む必要があります。
6	真偽値を表すには、trueまたはfalseを使用します。
7	特殊文字を表すには、 エスケープシーケンスを使用する必要があります。 例えば、改行を表すには、¥nを使用します。
8	文字列を複数行で表すには、" "または">"を使用します。 " "を使用すると、改行が文字列に反映されます。 ">"を使用すると、改行が文字列から削除されます。

```
# Playbookのyamlファイルサンプル (文法8箇条No5~8に対応)
- name: Setup nginx server on my list
  become: True
  tasks:
    - name: Ansible copy file to remote server
      copy:
        content: |
          <html>
          <body>
          <h1>Yamlサンプル</h1>
          </body>
          </html>
        dest: "/usr/share/nginx/html/index.html"

    - name: Run a script
      script: "#!/bin/bash\nncp index.html /html/"
```

Ansibleの基本3－モジュール

Ansibleの基本3－モジュール

- ・Playbookのタスク定義に使用するモジュール(タスク実施の為の部品)の種類と概要は以下の通り

No	種類	概要
1	標準モジュール	・Ansibleに標準で搭載されているモジュールであり、Ansibleをインストールすると即利用できる ・Ansible2.9では約430個の標準モジュールがあり、具体的には以下のようなものがある ・commandモジュール: ターゲットのホスト上で指定したシェルコマンドを実行する ・serviceモジュール: ターゲットのホスト上のサービスを管理(開始や終了等)する ・copyモジュール: マスターに配置されているファイルをターゲットの所定パスにコピーする ・メンテナンス内容に応じて使用するモジュールを選定してタスクを定義する
2	カスタムモジュール	・Ansibleに標準で搭載されていないモジュールであり、ユーザーが独自に作成し、作成したモジュールをPlaybook内で読み込むことで利用できる ・カスタムモジュールを作成する場合、使用するプログラミング言語に制限は無く、PythonやRuby、Go、Java、Perlなど、様々なプログラミング言語で作成することができる ・一方で、AnsibleはPythonを内部的に使用しているため、Pythonでカスタムモジュールを作成することが推奨されている ・標準モジュールで実施できない複雑なメンテナンスを行いたい場合に、自身でコードを記述しタスクに組み込む用途で使用する

Ansibleの基本3ーモジュールー

Ansibleの基本3ーモジュールー

・カスタムモジュール作成から利用の流れは以下の通り

No	作業	概要
1	カスタムモジュール作成	Python等を使用して複雑なメンテナンス内容をコード記述する
2	Ansible設定ファイル修正	Ansibleの設定ファイル（Ansibleで利用するインベントリの配置先パス等を定義）を修正し、カスタムモジュールの配置先やカスタムモジュール利用のために必要なライブラリの配置先を定義する <pre># カスタムモジュール配置先 library = /usr/share/my_modules/ # ライブラリ配置先 module_utils = /usr/lib/python2.7/site-packages/ansible/module_utils/</pre>
3	カスタムモジュール配置	No2で定義した配置先にカスタムモジュールを配置する
4	Playbook作成	Playbook内で作成したカスタムモジュール名を定義することでタスクにカスタムモジュールを組み込む
5	Playbook実行（カスタムモジュール利用）	Playbookを実行すると、カスタムモジュールが呼び出され、ターゲットに対し複雑なメンテナンスが行われる

Ansibleの基本4－ロール

ロール概要

- Ansibleのロールの概要は以下の通り

No	トピック	説明
1	ロールの概要	- Ansibleで実行されるタスクを整理するための仕組み - ロール使用前がさまざまなタスクをこなすゼネラリストとするなら、ロール使用後の各ロールは、特定の種類のタスクの実行を担うスペシャリストと言える
2	ロールの具体例	- Ansibleで実行されるタスクが、Webサーバーソフトウェアインストール、起動、DBサーバーソフトウェアインストール、起動の4つの場合、前半の2つ、後半の2つにタスク分割を行う - 前半の2つを「Webサーバー構築ロール」、後半の2つを「DBサーバー構築ロール」として実行できるようにする
3	ロールを使用するメリット	- Playbook (Yamlファイル) コード記述の生産性や品質向上が挙げられる - ロールが明確なら、コードの修正箇所の特定や修正を早く行え、また誤った修正を行うリスクを軽減することができる

Ansibleの基本4－ロール

ロール作成手順

- Ansibleのロール作成手順は以下の通り

No	手順	説明
1	タスク分割基準の決定	- タスク分割基準は大きく「リソースの種類」、「操作の種類」の2通りがある - リソースの種類基準とは、例えばWebサーバーソフトウェアのリソースに関連するタスクと、DBサーバーソフトウェアのリソースに関連するタスクを分けて管理することを指す - 操作の種類基準とは、例えばソフトウェアのインストール系のタスクと、起動系のタスクを分けて管理することを指す - どちらの基準を採用するかはプロジェクトで何を重視するかに応じて決定する（ロールを使用するメリットをより享受できるのはどちらかを考える、等）
2	タスク分割 & ロール名付与	既存のPlaybookからタスクを分割し、各タスクに対し実行するロール名を付与する（リソースの種類基準で分ける場合、「web_server_role」、「db_server_role」等）
3	各ロール呼び出し処理記述 (メインPlaybook)	No2で定義した各ロールを呼び出すためのPlaybookを作成する（本PlaybookをメインPlaybookと呼ぶ）

Ansibleの基本4ーロールー

ロール作成手順の具体例

- ・ 手順No2「タスク分割 & ロール名付与」の具体例は以下の通り

<既存Playbook>

```
# Playbook (1Yamlファイル)
- name: Setup web server and db server
  hosts:
    - devservers
  tasks:
    - name: Install the latest version of nginx
      command: amazon-linux-extras install nginx1 -y
    - name: Start nginx service
      service:
        name: nginx
        state: started

    - name: Install PostgreSQL
      yum:
        name: postgresql-server
        state: latest
    - name: Start PostgreSQL
      service:
        name: postgresql
        state: started
```

Ansibleの基本4ーロールー

ロール作成手順の具体例

- ・ 手順No2「タスク分割 & ロール名付与」の具体例は以下の通り

<既存Playbook>

```
# Playbook (1Yamlファイル)
- name: Setup web server and db server
  hosts:
    - devservers
  tasks:
    - name: Install the latest version of nginx
      command: amazon-linux-extras install nginx1 -y
    - name: Start nginx service
      service:
        name: nginx
        state: started
    - name: Install PostgreSQL
      yum:
        name: postgresql-server
        state: latest
    - name: Start PostgreSQL
      service:
        name: postgresql
        state: started
```

Webサーバーソフトウェア
インストール、起動タスク

DBサーバーソフトウェア
インストール、起動タスク

Ansibleの基本4ーロールー

ロール作成手順の具体例

- ・ 手順No2「タスク分割 & ロール名付与」の具体例は以下の通り

<既存Playbook>

```
# Playbook (1Yamlファイル)
- name: Setup web server and db server
  hosts:
    - devservers
  tasks:
```

```
- name: Install the latest version of nginx
  command: amazon-linux-extras install nginx1 -y
- name: Start nginx service
  service:
    name: nginx
    state: started
```

Webサーバーソフトウェア
インストール、起動タスク

```
- name: Install PostgreSQL
  yum:
    name: postgresql-server
    state: latest
- name: Start PostgreSQL
  service:
    name: postgresql
    state: started
```

DBサーバーソフトウェア
インストール、起動タスク

タスク分割 &
ロール名付与
(web_server_role)

```
# web_server_role (1Yamlファイル)
- name: Install the latest version of nginx
  command: amazon-linux-extras install nginx1 -y
- name: Start nginx service
  service:
    name: nginx
    state: started
```

タスク分割 &
ロール名付与
(db_server_role)

```
# db_server_role (1Yamlファイル)
- name: Install PostgreSQL
  yum:
    name: postgresql-server
    state: latest
- name: Start PostgreSQL
  service:
    name: postgresql
    state: started
```

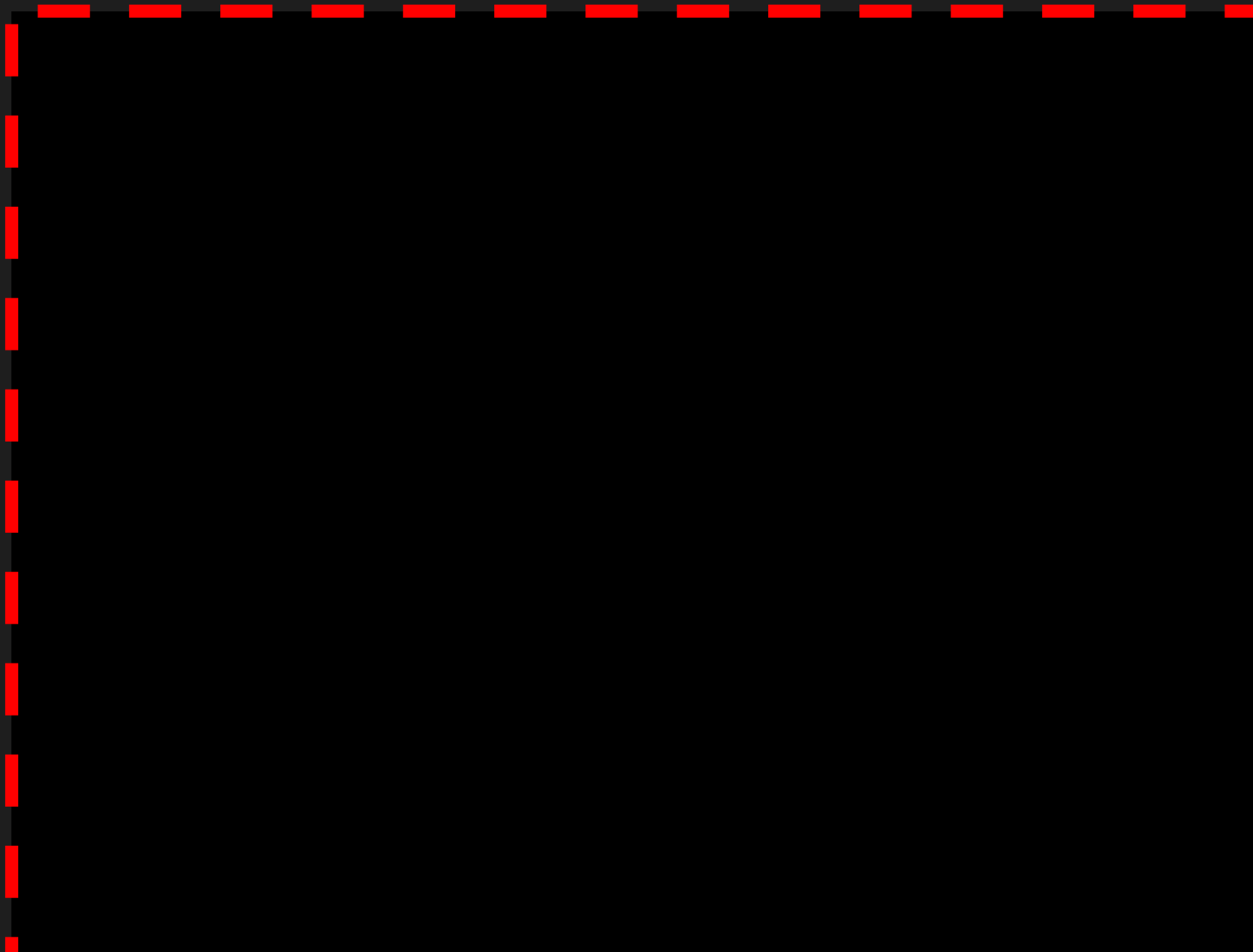
Ansibleの基本4ーロールー

ロール作成手順の具体例

- ・ 手順No3「各ロール呼び出し処理記述」の具体例は以下の通り

<既存Playbook>

```
# Playbook (1Yamlファイル)
- name: Setup web server and db server
  hosts:
    - devservers
  tasks:
```



Ansibleの基本4ーロールー

ロール作成手順の具体例

- ・ 手順No3「各ロール呼び出し処理記述」の具体例は以下の通り

<既存Playbook>

```
# Playbook (1Yamlファイル)
- name: Setup web server and db server
  hosts:
    - devservers
  tasks:
```

Ansibleの基本4ーロールー

ロール作成手順の具体例

- ・ 手順No3「各ロール呼び出し処理記述」の具体例は以下の通り

<既存Playbook>

```
# Playbook (1Yamlファイル)
```

```
name: Setup web server and db server
hosts:
  - devservers
```

```
tasks:
```

<メインPlaybook>

```
# Playbook (1Yamlファイル)
```

```
- name: Setup web server and db server
  hosts:
    - devservers
```

コピー

Ansibleの基本4ーロールー

ロール作成手順の具体例

- ・ 手順No3「各ロール呼び出し処理記述」の具体例は以下の通り

<既存Playbook>

```
# Playbook (1Yamlファイル)
- name: Setup web server and db server
  hosts:
    - devservers
  tasks:
```

各ロール呼び出し
処理記述

<メインPlaybook>

```
# Playbook (1Yamlファイル)
- name: Setup web server and db server
  hosts:
    - devservers
  roles:
    - web_server_role
    - db_server_role
```

Ansibleの基本5－出力結果の見方－

Ansibleの基本5－出力結果の見方－

- Ansibleの出力結果のサンプルは以下の通り

```
PLAY [devservers] *****

TASK [Install the latest version of nginx] *****
##実行結果メッセージ##
changed: [target-1]
##実行結果メッセージ##
changed: [target-2]

PLAY RECAP *****
target-1 : ok=1  changed=1  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
target-2 : ok=1  changed=1  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
```

Ansibleの基本5－出力結果の見方－

Ansibleの基本5－出力結果の見方－

・ Ansibleの出力結果の見方は以下の通り

1

```
PLAY [devservers] *****
```

2

```
TASK [Install the latest version of nginx] *****
##実行結果メッセージ##
changed: [target-1]
##実行結果メッセージ##
changed: [target-2]
```

```
PLAY RECAP *****
target-1 : ok=1  changed=1  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
target-2 : ok=1  changed=1  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
```

No	セクション	概要
1	PLAY	・どのターゲットに対し、playbookを実行するかを出力する(ターゲット指定に誤りが無いか要確認)
2	TASK	・Playbook内の各タスクごとに、タスク名とその実行結果を出力する ・タスクの実行結果に応じて、以下のステータスが出力される ・ok: 正常に終了した(ターゲットに変化無し) ・changed: 正常に終了した(ターゲットに変化有り) ・skipped: タスクを実行しなかった ・failed : エラーが発生した

Ansibleの基本5ー出力結果の見方ー

Ansibleの基本5ー出力結果の見方ー

- Ansibleの出力結果の見方は以下の通り

```
PLAY [devservers] *****

TASK [Install the latest version of nginx] *****
##実行結果メッセージ##
changed: [target-1]
##実行結果メッセージ##
changed: [target-2]

PLAY RECAP *****
target-1 : ok=1    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
target-2 : ok=1    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

3

No	セクション	概要
3	PLAY RECAP	▪ Playbook全体のタスク実行結果のサマリーを出力する ▪ 各項目の概要は以下の通り ▪ ok: 正常に終了したタスク数 ▪ changed: 正常に終了したタスクのうち、ターゲットの状態を変更したタスク数 ▪ unreachable: エラーが発生したタスク数(通信エラー起因) ▪ failed :エラーが発生したタスク数(通信エラー起因以外) ▪ skipped: タスクを実行しなかったタスク数 ▪ rescued: rescueディレクティブを実行したタスク数 ▪ ignored: エラーが発生したが無視したタスク数

AWSサービス・ツール紹介

AWSサービス・ツール紹介

・ハンズオンで使用するAWSサービス、ツールの概要は以下の通り

No	サービス名	概要
1	Amazon EC2	・AWS上に仮想的なサーバーを構築するためのサービス ・事前に作成したサブネットと紐付けて作成する ・EC2で作成したサーバーをEC2インスタンス、または単にインスタンスと呼ぶ ・OSは複数のAMI (Amazon Machine Image) からLinux用AMI、Windows用AMIなどから、またCPUやメモリ等も複数のインスタンスタイプから1vCPU, 1GiBメモリ、2vCPU, 4GiBメモリなどから用途に応じて選択することができる ・本講座では、Ansibleを使用してメンテナンスを行う対象の環境として使用する (Linux OSを使用)
2	Visual Studio Code	・Microsoftが開発したコード記述エディタ ・コードの補完機能や、文法エラーチェック機能など、さまざまな便利機能が備わっている ・本講座では、Ansibleのコード記述のために使用する
3	Tera Term	・サーバーにSSH接続するためのクライアントソフト ・本講座では、Ansibleを使用してメンテナンス後のEC2インスタンスにログインし、環境の変更が意図した通り行われているか確認するために使用する

ハンズオンの流れ

ハンズオンの流れ

・ハンズオンの流れは以下の通り

No	ステップ	対象レクチャー	概要
1	ハンズオン 事前準備	ハンズオン 事前準備 1～4	▪ AWS上にLinux OSのEC2インスタンスを作成（マスター×1、ターゲット×4） ▪ マスターにAnsibleをインストール ▪ VSCodeの環境設定（開発用ディレクトリ作成）
2	インベントリ の使い方確認	ハンズオン 1～2	▪ 標準モジュール（commandモジュール、serviceモジュール）を使用したPlaybook作成（ターゲットにNginxインストール、自動起動設定等実施） ▪ Hostsファイルを使用したPlaybook実行確認 ▪ インベントリファイルを使用したPlaybook実行確認 ▪ エラー（Unreachable）発生条件確認

ハンズオンの流れ

ハンズオンの流れ

・ハンズオンの流れは以下の通り

No	ステップ	対象レクチャー	概要
3	標準 モジュール の使い方確認	ハンズオン 3～22	・ハンズオン1～2で構築したWebサーバーに対し、各種モジュールを使用しながら更なるメンテナンスを実施 (ファイルコピー、ファイル変更、リンク押下、サーバー再起動、ユーザー作成、Webサーバー環境設定変更等) ・合計11の標準モジュールの使い方確認 ・Playbook内で利用する変数 (vars変数、register変数)の使い方確認

ハンズオンの流れ

ハンズオンの流れ

・ハンズオンの流れは以下の通り

No	ステップ	対象レクチャー	概要
4	カスタム モジュール の使い方確認	ハンズオン 23	・ハンズオン3～22で構築したWebサイトに対し、カスタムモジュールを使用しWebページ修正（ページ内小文字の大文字化）を実施 ・カスタムモジュール作成（Pythonコード作成）、カスタムモジュール利用のための環境設定、Playbook実行確認 ・エラー（Syntax error）発生条件確認

ハンズオンの流れ

ハンズオンの流れ

・ハンズオンの流れは以下の通り

No	ステップ	対象レクチャー	概要
5	ロール の使い方確認	ハンズオン 24～25	・ロール分割前のPlaybookとして、Webサーバー（Nginx）、DBサーバー（PostgreSQL）のインストール、自動起動設定等のタスクを定義し作成、Playbook実行確認 ・ロール分割前のPlaybookを各ロール（Webサーバー構築、DBサーバー構築）に分割し作成、Playbook実行確認 ・標準モジュール（yumモジュール）の実行確認
6	補足説明 （EIP割り当て）	ハンズオン 補足	・EC2インスタンスのIPアドレス固定化手順（EC2インスタンスはデフォルトでサーバ終了後、起動時にIPアドレスが変更されるが、この固定化方法の確認）